DR. RVR NRI INSTITUTE OF TECHNOLOGY

An Institution Deemed to be University u/s 3 of UGC Act, 1956 | Pothavarappadu (V), Agiripalli (M), Eluru District, A.P. - 521212

School of Computer Studies — Department of I B.Tech Python Programming

Python Programming – Activity-Based Learning (ABL) Consolidated Report

Comprehensive documentation of 10 selected case studies implemented strictly using core syllabus constructs: variables, operators, conditional statements, loops, strings, lists, nested lists, tuples, and tuple unpacking.

Student Name:	Benny Bobbili	**Roll Number:**	[26000861]
Program / Year:	I B.Tech (Semester I)	**Branch & Section:**	CSM – [1]
Course Title:	Python Programming	**GitHub Repo:**	Python-Activity-Based-Learning

INDEX OF 10 SELECTED CASE STUDIES

S.No	Problem #	Case Study Title	Core Syllabus Concepts Used	Page No
01	Case #08	Manufacturing Cost and Profitability Analysis	Variables, Float, Arithmetic, if-elif-else	Page 2
02	Case #12	Attendance Recovery Computation	Percentages, while Loop, Conditionals	Page 3
03	Case #18	Fuel Consumption and Route Comparison	for Loop, Minimum Tracking, Operators	Page 4
04	Case #33	Campus Registration Identifier Analyzer	Strings, Slicing, isdigit(), isalpha()	Page 5
05	Case #39	Multi-Stage Examination Eligibility System	Multiple Independent if Checks, Flags	Page 6
06	Case #45	Weekly Electricity Consumption Matrix	2D Nested Lists, Row & Column Sums	Page 7
07	Case #49	Tuple-Based Product Catalogue Audit	List of Tuples, Tuple Unpacking, Lists	Page 8
08	Case #51	Bus Passenger Occupancy Simulation	Sequential for Loop, State Tracking	Page 9
09	Case #53	Food Waste Monitoring System	Lists, Percentage Calculation, Loops	Page 10
10	Case #59	Campus Transport Allocation	Integer Division, Fleet Allocation Loop	Page 11

Declaration & Deliverable Checklist: Each case study on the following pages includes all 9 mandatory deliverables: (1) Problem Statement, (2) Objectives, (3) Assumptions, (4) Algorithm, (5) Flowchart, (6) Pseudocode, (7) Python Source Code, (8) Test Cases & Output Screenshot Log, and (9) Conclusion.

Case Study 01: Manufacturing Cost and Profitability Analysis

Problem #8

1. PROBLEM STATEMENT

Calculate material costs, labour charges, wastage, machine costs, and selling prices; identify whether production meets a specified profit target.

2. OBJECTIVES

To compute total production cost and net profit using arithmetic operators, and verify whether achieved profit meets the target using conditional statements.

3. ASSUMPTIONS

All cost inputs and selling prices are positive values. Total cost is the exact sum of material, labour, wastage, and machine expenses.

4. ALGORITHM

1. Start the program.
2. Read `mat`, `lab`, `wast`, `mach`, `sell`, and `target`.
3. Compute `total_cost = mat + lab + wast + mach`.
4. Compute `profit = sell - total_cost`.
5. If `profit ≥ target`, print target met; elif `profit > 0`, print shortfall; else print loss.
6. Stop the program.

6. PSEUDOCODE

```
BEGIN
  INPUT mat, lab, wast, mach, sell, target
  total_cost = mat + lab + wast + mach
  profit = sell - total_cost
  IF profit >= target THEN
    PRINT "Profit target MET"
  ELSE IF profit > 0 THEN
    PRINT "Shortfall:", target - profit
  ELSE
    PRINT "Loss incurred"
  END IF
END
```

5. FLOWCHART



7. PYTHON SOURCE CODE (CASE_STUDY_01_MANUFACTURING_PROFIT.PY)

```
# Case Study 8: Manufacturing Cost & Profitability
mat = float(input("Enter material cost: "))
lab = float(input("Enter labour charges: "))
wast = float(input("Enter wastage cost: "))
mach = float(input("Enter machine cost: "))
sell = float(input("Enter total selling price: "))
target = float(input("Enter profit target: "))

total_cost = mat + lab + wast + mach
profit = sell - total_cost

print("\n--- Profitability Report ---")
print("Total Production Cost :", total_cost)
print("Selling Price         :", sell)
print("Actual Profit / Loss  :", profit)

if profit >= target:
    print("Status: Profit target MET successfully!")
elif profit > 0:
    shortfall = target - profit
    print("Status: Profit BELOW target by:", shortfall)
else:
    print("Status: Production resulted in a LOSS.")
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter material cost: 5000
Enter labour charges: 2000
Enter wastage cost: 500
Enter machine cost: 1500
Enter total selling price: 12000
Enter profit target: 2500

--- Profitability Report ---
Total Production Cost : 9000.0
Selling Price         : 12000.0
Actual Profit / Loss  : 3000.0
Status: Profit target MET
successfully!

Process finished with exit code 0
```

9. Conclusion: The program accurately computes total manufacturing costs from four expense heads and evaluates whether the achieved profit meets the target threshold using conditional branching.

Case Study 02: Attendance Recovery Computation

Problem #12

1. PROBLEM STATEMENT

Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold under future attendance scenarios.

2. OBJECTIVES

To calculate initial attendance percentage and use an iterative `while` loop to find the exact number of consecutive future classes needed to reach eligibility.

3. ASSUMPTIONS

Total classes held is greater than zero, the eligibility threshold is less than 100% (typically 75%), and each attended future class increments both numerator and denominator by 1.

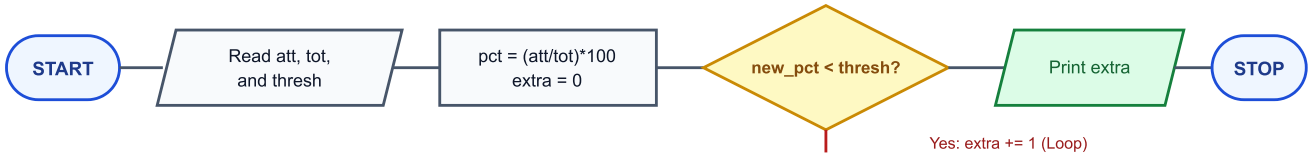
4. ALGORITHM

1. Start the program.
2. Read `att` (attended), `tot` (total held), and `thresh`.
3. Compute `curr_pct = (att / tot) * 100`.
4. Initialize `extra = 0`.
5. While `((att + extra) / (tot + extra)) * 100 < thresh`, increment `extra = extra + 1`.
6. Display `curr_pct` and `extra`, then stop.

6. PSEUDOCODE

```
BEGIN
  INPUT att, tot, thresh
  curr_pct = (att / tot) * 100
  SET extra = 0
  WHILE ((att + extra) / (tot + extra)) * 100
< thresh DO
    extra = extra + 1
  END WHILE
  PRINT curr_pct, extra
END
```

5. FLOWCHART



7. PYTHON SOURCE CODE (CASE_STUDY_02_ATTENDANCE_RECOVERY.PY)

```
# Case Study 12: Attendance Recovery Computation
att = int(input("Enter classes attended so far: "))
tot = int(input("Enter total classes held so far: "))
thresh = float(input("Enter required percentage (e.g.,
75): "))

curr_pct = (att / tot) * 100
print("\nCurrent Attendance:", round(curr_pct, 2), "%")

if curr_pct >= thresh:
    print("Status: Already eligible! Extra classes
needed: 0")
else:
    extra = 0
    while ((att + extra) / (tot + extra)) * 100 <
thresh:
        extra = extra + 1

    new_pct = ((att + extra) / (tot + extra)) * 100
    print("Minimum consecutive classes needed:", extra)
    print("Updated Attendance:", round(new_pct, 2), "%")
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter classes attended so far: 30
Enter total classes held so far:
50
Enter required percentage (e.g.,
75): 75

Current Attendance: 60.0 %
Minimum consecutive classes
needed: 30
Updated Attendance: 75.0 %
```

Process finished with exit code 0

9. Conclusion: The program computes current attendance percentage and uses a `while` loop to determine the exact number of consecutive future classes required to achieve eligibility.

Case Study 03: Fuel Consumption and Route Comparison

Problem #18

1. PROBLEM STATEMENT

Calculate fuel requirements and journey costs for several vehicles using distance, mileage, fuel prices, and reserve requirements; identify the lowest-cost feasible option.

2. OBJECTIVES

To compute fuel consumption (including mandatory reserve fuel) across multiple vehicles using a `for` loop and determine the lowest-cost vehicle.

3. ASSUMPTIONS

Distance, mileage (km/l), and fuel price per liter are positive numbers. Total fuel required equals `(distance / mileage) + reserve_fuel`.

4. ALGORITHM

1. Start and read `dist`, `price`, and vehicle count n.
2. Set `min_cost = 9999999.0` and `best_veh = ""`.
3. Repeat for `i = 1` to n: read `v_name`, `mil`, and `res`.
4. Compute `fuel = (dist / mil) + res` and `cost = fuel * price`.
5. If `cost < min_cost`, update `min_cost` and `best_veh`.
6. Print `best_veh` and `min_cost`, then stop.

6. PSEUDOCODE

```
BEGIN
  INPUT dist, price, n
  SET min_cost = 9999999, best_veh = ""
  FOR i FROM 1 TO n DO
    INPUT v_name, mil, res
    fuel = (dist / mil) + res
    cost = fuel * price
    IF cost < min_cost THEN
      min_cost = cost
      best_veh = v_name
    END IF
  END FOR
  PRINT best_veh, min_cost
END
```

5. FLOWCHART



7. PYTHON SOURCE CODE (CASE_STUDY_03_FUEL_COMPARISON.PY)

```
# Case Study 18: Fuel Consumption & Route Comparison
dist = float(input("Enter journey distance (km): "))
price = float(input("Enter fuel price per liter (Rs):
"))
n = int(input("Enter number of vehicles to compare: "))

min_cost = 9999999.0
best_veh = ""

for i in range(1, n + 1):
    print("\n--- Vehicle", i, "---")
    v_name = input("Enter vehicle name: ")
    mil = float(input("Enter mileage (km/l): "))
    res = float(input("Enter reserve fuel (liters): "))

    fuel_needed = (dist / mil) + res
    trip_cost = fuel_needed * price
    print("Fuel:", round(fuel_needed, 2), "L | Cost:
Rs.", round(trip_cost, 2))

    if trip_cost < min_cost:
        min_cost = trip_cost
        best_veh = v_name

print("\n=== Best Option ===")
print("Lowest-Cost Vehicle :", best_veh)
print("Minimum Journey Cost: Rs.", round(min_cost, 2))
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter journey distance (km): 300
Enter fuel price per liter (Rs):
100
Enter number of vehicles to
compare: 2

--- Vehicle 1 ---
Enter vehicle name: Car
Enter mileage (km/l): 15
Enter reserve fuel (liters): 5
Fuel: 25.0 L | Cost: Rs. 2500.0

--- Vehicle 2 ---
Enter vehicle name: Bike
Enter mileage (km/l): 50
Enter reserve fuel (liters): 2
Fuel: 8.0 L | Cost: Rs. 800.0

=== Best Option ===
Lowest-Cost Vehicle : Bike
Minimum Journey Cost: Rs. 800.0

Process finished with exit code 0
```

Case Study 04: Campus Registration Identifier Analyzer

Problem #33

1. PROBLEM STATEMENT

Examine student identification strings, validate their structure, extract specified components, and classify records according to branch and admission year codes.

2. OBJECTIVES

To validate the length and character structure of student registration strings using string slicing and built-in string methods (`isdigit()`, `isalpha()`).

3. ASSUMPTIONS

Valid registration IDs follow the 7-character pattern YYBBBNN (e.g., 26CSE45), where YY is admission year, BBB is branch code, and NN is roll number.

4. ALGORITHM

1. Start and read count of student IDs n.
2. Repeat n times: read `reg` and convert to uppercase.
3. If `len(reg) != 7`, print invalid length error.
4. Slice `yr = reg[0:2]`, `br = reg[2:5]`, `num = reg[5:7]`.
5. If `yr.isdigit()`, `br.isalpha()`, and `num.isdigit()`, print extracted fields; else flag malformed structure.
6. Stop the program.

6. PSEUDOCODE

```
BEGIN
  INPUT n
  FOR i FROM 1 TO n DO
    INPUT reg
    IF LENGTH(reg) == 7 THEN
      yr = reg[0:2], br = reg[2:5], num =
reg[5:7]
      IF yr IS DIGIT AND br IS ALPHA AND num
IS DIGIT THEN
        PRINT "Valid:", yr, br, num
      ELSE PRINT "Invalid character groups"
    ELSE PRINT "Invalid length"
  END FOR
END
```

5. FLOWCHART



7. PYTHON SOURCE CODE (CASE_STUDY_04_REGISTRATION_ANALYZER.PY)

```
# Case Study 33: Campus Registration Identifier Analyzer
n = int(input("Enter number of student IDs to analyze:
"))

for i in range(1, n + 1):
    reg = input("\nEnter ID (YYBBBNN, e.g., 26CSE45):
").upper()

    if len(reg) != 7:
        print("Status: INVALID (Length must be 7
characters)")
    else:
        yr = reg[0:2]
        br = reg[2:5]
        num = reg[5:7]

        if yr.isdigit() and br.isalpha() and
num.isdigit():
            print("Status         : VALID")
            print("Admission Year : 20" + yr)
            print("Branch Code    :", br)
            print("Student Number :", num)
        else:
            print("Status: INVALID (Expected 2 digits, 3
letters, 2 digits)")
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter number of student IDs to
analyze: 2

Enter ID (YYBBBNN, e.g.,
26CSE45): 26CSE45
Status         : VALID
Admission Year : 2026
Branch Code    : CSE
Student Number : 45

Enter ID (YYBBBNN, e.g.,
26CSE45): 26CS01
Status: INVALID (Length must be 7
characters)

Process finished with exit code 0
```

9. Conclusion: The program validates student registration identifiers using string slicing and character-category checks,

Case Study 05: Multi-Stage Examination Eligibility System

Problem #39

1. PROBLEM STATEMENT

Determine eligibility for a final examination using attendance, laboratory completion, internal assessment, and fee-clearance conditions, reporting every unmet requirement.

2. OBJECTIVES

To evaluate four independent academic and administrative conditions using separate `if` statements so that every unmet requirement is reported clearly.

3. ASSUMPTIONS

Eligibility requires: Attendance ≥ 75%, Lab Completed = "YES", Internal Marks ≥ 20 (out of 40), and Tuition Fee Cleared = "YES".

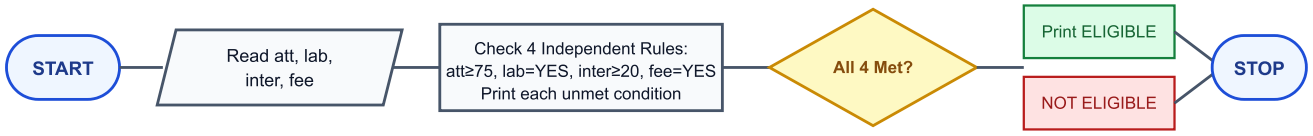
4. ALGORITHM

1. Start and read `name`, `att`, `lab`, `inter`, and `fee`.
2. Initialize boolean flag `eligible = True`.
3. Check `att < 75`, `lab != "YES"`, `inter < 20`, and `fee != "YES"` using independent `if` blocks.
4. For any failed check, print the specific unmet reason and set `eligible = False`.
5. Print final eligibility status and stop.

6. PSEUDOCODE

```
BEGIN
  INPUT name, att, lab, inter, fee
  SET eligible = TRUE
  IF att < 75 THEN PRINT "Low Attendance",
eligible = FALSE
  IF lab != "YES" THEN PRINT "Lab
Incomplete", eligible = FALSE
  IF inter < 20 THEN PRINT "Low Internals",
eligible = FALSE
  IF fee != "YES" THEN PRINT "Fee Pending",
eligible = FALSE
  IF eligible == TRUE THEN PRINT "ELIGIBLE"
  ELSE PRINT "NOT ELIGIBLE"
END
```

5. FLOWCHART



7. PYTHON SOURCE CODE (CASE_STUDY_05_EXAM_ELIGIBILITY.PY)

```
# Case Study 39: Multi-Stage Examination Eligibility
name = input("Enter Student Name: ")
att = float(input("Enter Attendance Percentage: "))
lab = input("Is Laboratory Completed? (YES/NO):
").upper()
inter = float(input("Enter Internal Marks (out of 40):
"))
fee = input("Is Tuition Fee Cleared? (YES/NO):
").upper()

print("\n--- Eligibility Audit for", name, "---")
eligible = True

if att < 75.0:
    print("- Unmet: Attendance is below 75%")
    eligible = False
if lab != "YES":
    print("- Unmet: Laboratory work is incomplete")
    eligible = False
if inter < 20.0:
    print("- Unmet: Internal marks are below 20")
    eligible = False
if fee != "YES":
    print("- Unmet: Tuition fee is not cleared")
    eligible = False

if eligible == True:
    print("Final Status: ELIGIBLE for Final Exam.")
else:
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter Student Name: Rahul
Enter Attendance Percentage: 70
Is Laboratory Completed?
(YES/NO): YES
Enter Internal Marks (out of 40):
25
Is Tuition Fee Cleared? (YES/NO):
NO

--- Eligibility Audit for Rahul -
--
- Unmet: Attendance is below 75%
- Unmet: Tuition fee is not
cleared
Final Status: NOT ELIGIBLE for
Final Exam.
```

Case Study 06: Weekly Electricity Consumption Matrix

Problem #45

1. PROBLEM STATEMENT

Process appliance-wise electricity readings across seven days, calculate daily and appliance totals, and identify consumption exceeding specified limits.

2. OBJECTIVES

To represent a 2D matrix of electricity usage using nested lists, calculate row-wise (appliance) and column-wise (daily) totals, and flag limit violations.

3. ASSUMPTIONS

Readings are recorded in kWh units for n appliances over 7 days. Any appliance whose 7-day sum exceeds `limit` is flagged.

4. ALGORITHM

1. Start and read number of appliances n and weekly `limit`.
2. Use nested loops (n rows × 7 days) to populate 2D list `grid`.
3. For each appliance `i`, sum `grid[i][0..6]` and check if total > `limit`.
4. For each day d from 0 to 6, sum `grid[0..n-1][d]` and print daily total.
5. Stop the program.

6. PSEUDOCODE

```
BEGIN
  INPUT n, limit
  READ n x 7 matrix into nested list grid
  FOR i FROM 0 TO n - 1 DO
    app_tot = SUM(grid[i][0..6])
    IF app_tot > limit THEN PRINT "Exceeded"
  END FOR
  FOR d FROM 0 TO 6 DO
    day_tot = SUM(grid[0..n-1][d])
    PRINT "Day", d+1, day_tot
  END FOR
END
```

5. FLOWCHART



7. PYTHON SOURCE CODE (CASE_STUDY_06_ELECTRICITY_MATRIX.PY)

```
# Case Study 45: Weekly Electricity Consumption Matrix
n = int(input("Enter number of appliances: "))
limit = float(input("Enter weekly limit per appliance:
"))
grid = []

for i in range(n):
    print("\nEnter 7 days readings for Appliance", i +
1, ":")
    row = []
    for d in range(7):
        u = float(input("  Day " + str(d + 1) + ": "))
        row.append(u)
    grid.append(row)

print("\n--- Appliance-Wise Weekly Totals ---")
for i in range(n):
    app_tot = 0.0
    for d in range(7):
        app_tot = app_tot + grid[i][d]
    if app_tot > limit:
        print("Appliance", i + 1, ":", app_tot, "
[EXCEEDED LIMIT!]")
    else:
        print("Appliance", i + 1, ":", app_tot, "[Within
Limit]")

print("\n--- Day-Wise Total Consumption ---")
for d in range(7):
    day_tot = 0.0
    for i in range(n):
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter number of appliances: 2
Enter weekly limit per appliance:
30
Appliance 1 (7 days): 5, 5, 5, 5,
5, 5, 5
Appliance 2 (7 days): 2, 2, 2, 2,
2, 2, 2

--- Appliance-Wise Weekly Totals
---
Appliance 1 : 35.0 [EXCEEDED
LIMIT!]
Appliance 2 : 14.0 [Within Limit]

--- Day-Wise Total Consumption --
-
Day 1 Total: 7.0 units
Day 2 Total: 7.0 units
...
Day 7 Total: 7.0 units
```

Case Study 07: Tuple-Based Product Catalogue Audit

Problem #49

1. PROBLEM STATEMENT

Store product identifiers, prices, and stock quantities in tuples; detect invalid records, calculate inventory values, and identify items requiring replenishment.

2. OBJECTIVES

To store product data as immutable tuples inside a list, unpack tuple elements in a loop, flag invalid prices/quantities, and compute total inventory valuation.

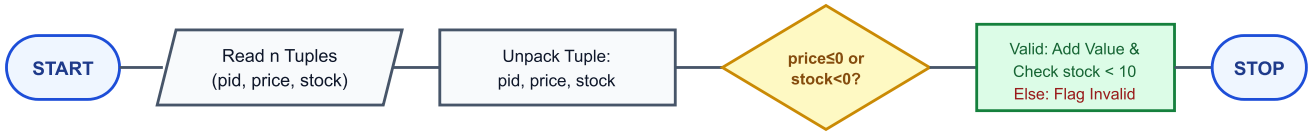
3. ASSUMPTIONS

A product record is invalid if `price ≤ 0` or `stock < 0`. Any valid product with `stock < 10` is flagged for replenishment.

4. ALGORITHM

1. Start and read number of products n.
2. Read `(pid, price, stock)` for each product and append tuple to `catalogue`.
3. Loop through `catalogue` and unpack `pid, price, stock`.
4. If `price ≤ 0` or `stock < 0`, add `pid` to `invalid_items`.
5. Else add `price * stock` to `total_val`; if `stock < 10`, add to `replenish_items`.
6. Print summary and stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  INPUT n
  READ n tuples (pid, price, stock) into
catalogue
  SET total_val = 0
  FOR each (pid, price, stock) IN catalogue
DO
    IF price <= 0 OR stock < 0 THEN
      APPEND pid TO invalid_items
    ELSE
      total_val = total_val + (price * stock)
      IF stock < 10 THEN APPEND pid TO
replenish_items
    END IF
  END FOR
  PRINT total_val, invalid_items,
replenish_items
END
```

7. PYTHON SOURCE CODE (CASE_STUDY_07_PRODUCT_CATALOGUE_AUDIT.PY)

```
# Case Study 49: Tuple-Based Product Catalogue Audit
n = int(input("Enter number of products: "))
catalogue = []

for i in range(n):
    pid = input("\nEnter Product ID: ").upper()
    price = float(input("Enter Unit Price: "))
    stock = int(input("Enter Stock Quantity: "))
    catalogue.append((pid, price, stock))

total_val = 0.0
invalid_items = []
replenish_items = []

for i in range(n):
    pid, price, stock = catalogue[i]
    if price <= 0 or stock < 0:
        invalid_items.append(pid)
    else:
        total_val = total_val + (price * stock)
        if stock < 10:
            replenish_items.append(pid)

print("\n=== Catalogue Audit Summary ===")
print("Total Inventory Value :", total_val)
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter number of products: 3

Enter Product ID: P101
Enter Unit Price: 500
Enter Stock Quantity: 20

Enter Product ID: P102
Enter Unit Price: 200
Enter Stock Quantity: 5

Enter Product ID: P103
Enter Unit Price: -100
Enter Stock Quantity: 10

=== Catalogue Audit Summary ===
Total Inventory Value : 11000.0
Invalid Records       : ['P103']
Replenish Needed (<10): ['P102']
```

Case Study 08: Bus Passenger Occupancy Simulation

Problem #51

1. PROBLEM STATEMENT

Process boarding and alighting counts across multiple stops, calculate passenger counts at each stop, and flag any capacity violations.

2. OBJECTIVES

To simulate passenger count changes across sequential bus stops using a `for` loop and detect overcrowding whenever occupancy exceeds bus capacity.

3. ASSUMPTIONS

The bus starts empty (0 passengers) before Stop 1. At each stop, updated occupancy equals `previous + boarding - alighting`.

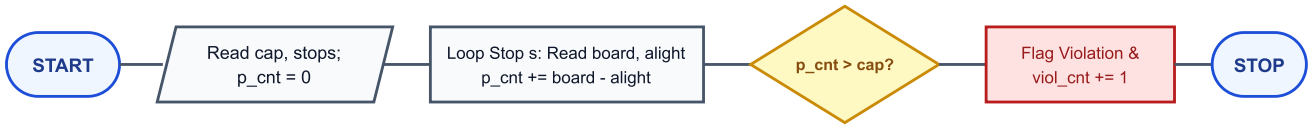
4. ALGORITHM

1. Start and read bus `cap` and number of `stops`.
2. Initialize `p_cnt = 0` and `viol_cnt = 0`.
3. Loop `s` from 1 to `stops`: read `board` and `alight`.
4. Update `p_cnt = p_cnt + board - alight`.
5. If `p_cnt > cap`, print overcrowding alert and increment `viol_cnt`.
6. Print final passenger count and total violations, then stop.

6. PSEUDOCODE

```
BEGIN
  INPUT cap, stops
  SET p_cnt = 0, viol_cnt = 0
  FOR s FROM 1 TO stops DO
    INPUT board, alight
    p_cnt = p_cnt + board - alight
    PRINT "Stop", s, "Occupancy:", p_cnt
    IF p_cnt > cap THEN
      PRINT "Violation!"
      viol_cnt = viol_cnt + 1
    END IF
  END FOR
  PRINT p_cnt, viol_cnt
END
```

5. FLOWCHART



7. PYTHON SOURCE CODE (CASE_STUDY_08_BUS_OCCUPANCY.PY)

```
# Case Study 51: Bus Passenger Occupancy Simulation
cap = int(input("Enter maximum bus seating capacity: "))
stops = int(input("Enter number of bus stops: "))

p_cnt = 0
viol_cnt = 0

for s in range(1, stops + 1):
    print("\n--- Stop", s, "---")
    board = int(input("Enter passengers boarding: "))
    alight = int(input("Enter passengers alighting: "))

    p_cnt = p_cnt + board - alight
    if p_cnt < 0:
        p_cnt = 0

    print("Passengers in bus at Stop", s, ":", p_cnt)
    if p_cnt > cap:
        print("ALERT: Overloaded by", p_cnt - cap,
"passengers!")
        viol_cnt = viol_cnt + 1

print("\n=== Simulation Summary ===")
print("Final Passengers on Bus   :", p_cnt)
print("Total Capacity Violations :", viol_cnt)
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter maximum bus seating
capacity: 40
Enter number of bus stops: 3

--- Stop 1 ---
Enter passengers boarding: 30
Enter passengers alighting: 0
Passengers in bus at Stop 1 : 30

--- Stop 2 ---
Enter passengers boarding: 20
Enter passengers alighting: 5
Passengers in bus at Stop 2 : 45
ALERT: Overloaded by 5
passengers!

--- Stop 3 ---
Enter passengers boarding: 2
Enter passengers alighting: 15
Passengers in bus at Stop 3 : 32

=== Simulation Summary ===
Final Passengers on Bus   : 32
Total Capacity Violations : 1

Process finished with exit code 0
```

Case Study 09: Food Waste Monitoring System

Problem #53

1. PROBLEM STATEMENT

Compare prepared meals, meals served, and leftovers across several days; calculate wastage percentages and identify days exceeding acceptable limits.

2. OBJECTIVES

To compute daily leftover meal counts and food wastage percentages across multiple days and record days that exceed the acceptable threshold in a list.

3. ASSUMPTIONS

Meals prepared is greater than zero and greater than or equal to meals served. Leftover meals equal `prepared - served`.

4. ALGORITHM

1. Start and read `days` and acceptable percentage `limit`.
2. Initialize empty list `high_waste_days = []`.
3. Loop d from 1 to `days`: read `prep` and `serv`.
4. Compute `leftover = prep - serv` and `waste_pct = (leftover / prep) * 100`.
5. If `waste_pct > limit`, append d to `high_waste_days`.
6. Display `high_waste_days` and stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  INPUT days, limit
  SET high_waste_days = []
  FOR d FROM 1 TO days DO
    INPUT prep, serv
    leftover = prep - serv
    waste_pct = (leftover / prep) * 100
    IF waste_pct > limit THEN
      APPEND d TO high_waste_days
    END IF
  END FOR
  PRINT high_waste_days
END
```

7. PYTHON SOURCE CODE (CASE_STUDY_09_FOOD_WASTE_MONITOR.PY)

```
# Case Study 53: Food Waste Monitoring System
days = int(input("Enter number of days to monitor: "))
limit = float(input("Enter acceptable wastage % limit:
"))

high_waste_days = []

for d in range(1, days + 1):
    print("\n--- Day", d, "---")
    prep = int(input("Enter meals prepared: "))
    serv = int(input("Enter meals served: "))

    leftover = prep - serv
    waste_pct = (leftover / prep) * 100
    print("Leftover:", leftover, "| Wastage:",
round(waste_pct, 2), "%")

    if waste_pct > limit:
        print("Warning: Wastage exceeded acceptable
limit!")
        high_waste_days.append(d)

print("\n=== Monitoring Summary ===")
print("Days Exceeding Wastage Limit:", high_waste_days)
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter number of days to monitor:
3
Enter acceptable wastage % limit:
10

--- Day 1 ---
Enter meals prepared: 200
Enter meals served: 190
Leftover: 10 | Wastage: 5.0 %

--- Day 2 ---
Enter meals prepared: 200
Enter meals served: 160
Leftover: 40 | Wastage: 20.0 %
Warning: Wastage exceeded
acceptable limit!

--- Day 3 ---
Enter meals prepared: 150
Enter meals served: 140
Leftover: 10 | Wastage: 6.67 %

=== Monitoring Summary ===
Days Exceeding Wastage Limit: [2]

Process finished with exit code 0
```

9. Conclusion: The program computes daily food wastage percentages from prepared and served meal counts and stores

Case Study 10: Campus Transport Allocation

Problem #59

1. PROBLEM STATEMENT

Determine the number of buses needed for several routes, account for bus capacities and passenger demand, and identify unserved passengers.

2. OBJECTIVES

To calculate buses required per route using ceiling integer division, allocate buses from a limited fleet, and track unserved passengers when fleet capacity runs out.

3. ASSUMPTIONS

Each bus has uniform capacity `cap`. Buses required for demand passengers is `(demand + cap - 1) // cap`.

4. ALGORITHM

1. Start and read bus `cap`, available `fleet`, and `routes`.
2. Loop each route: read `r_name` and passenger demand.
3. Compute `needed = (demand + cap - 1) // cap`.
4. If `needed ≤ fleet`, set `alloc = needed`, `unserved = 0`, and reduce `fleet`.
5. Else set `alloc = fleet`, `unserved = demand - (alloc * cap)`, and `fleet = 0`.
6. Print route allocation and total unserved passengers, then stop.

5. FLOWCHART



6. PSEUDOCODE

```
BEGIN
  INPUT cap, fleet, routes
  FOR r FROM 1 TO routes DO
    INPUT r_name, demand
    needed = (demand + cap - 1) DIV cap
    IF needed <= fleet THEN
      alloc = needed, unserved = 0, fleet =
fleet - alloc
    ELSE
      alloc = fleet, unserved = demand -
(alloc * cap), fleet = 0
    END IF
    PRINT r_name, needed, alloc, unserved
  END FOR
END
```

7. PYTHON SOURCE CODE (CASE_STUDY_10_TRANSPORT_ALLOCATION.PY)

```
# Case Study 59: Campus Transport Allocation
cap = int(input("Enter seating capacity per bus: "))
fleet = int(input("Enter total buses available in fleet:
"))
routes = int(input("Enter number of routes: "))

total_unserved = 0

for r in range(1, routes + 1):
    print("\n--- Route", r, "---")
    r_name = input("Enter Route Name: ")
    demand = int(input("Enter passenger demand: "))

    needed = (demand + cap - 1) // cap
    if needed <= fleet:
        alloc = needed
        unserved = 0
        fleet = fleet - alloc
    else:
        alloc = fleet
        unserved = demand - (alloc * cap)
        fleet = 0

    total_unserved = total_unserved + unserved
    print("Route:", r_name, "| Needed:", needed, "|
Alloc:", alloc, "| Unserved:", unserved)

print("\n=== Transport Summary ===")
```

8. TEST CASE & OUTPUT SCREENSHOT LOG

```
Enter seating capacity per bus:
50
Enter total buses available in
fleet: 4
Enter number of routes: 2

--- Route 1 ---
Enter Route Name: Vijayawada
Enter passenger demand: 120
Route: Vijayawada | Needed: 3 |
Alloc: 3 | Unserved: 0

--- Route 2 ---
Enter Route Name: Guntur
Enter passenger demand: 80
Route: Guntur | Needed: 2 |
Alloc: 1 | Unserved: 30

=== Transport Summary ===
Remaining Fleet Buses     : 0
Total Unserved Passengers : 30
```